



CS-UY 1134

Data Structures & Algorithms

Spring 2026
Introduction

Prof. Romero Cruz

Sebastián Romero Cruz

"Prof. Romero Cruz"



(they / them)

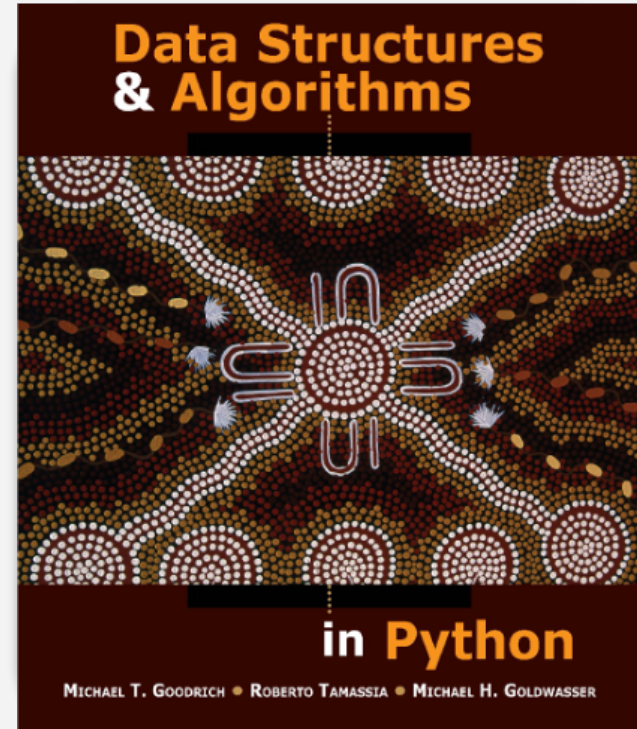
Industry Assistant Professor

Office Hours

- **Mondays / Wednesdays:** 12:30pm - 3:00pm (in-person & Zoom)
- To participate in my office hours (in-person or via Zoom), please book a 30 minute appointment via [my scheduling system](#).
- If these times do not work with your schedule, please email me and suggest times you are available to meet and I will check my schedule and get back to you on an agreeable time.

Textbook

"Data Structures & Algorithms in Python", 1st Edition, Goodrich, Tamassia, Goldwasser



Final Grade

30%	Highest exam grade
25%	Second highest exam grade
20%	Lowest exam grade
10%	Homework
15%	Lab Grade

Required tools

- Python 3 (3.10)
- Python IDE
 - PyCharm
 - Idle
 - Thonny

Review

- Data
 - Built-in types
 - int, float, bool, str
 - list, tuple, dict
- Expressions
 - I/O Expressions
 - Assignment
 - Arithmetic Expressions
 - Boolean Expressions
- Programmer-defined types
 - Classes
- Control flow
 - Sequential
 - Branching
 - if, elif, else
 - Iterative
 - while, for
 - Procedural (functions)
 - Exceptions / exception handling

List Mutation vs. Construction

Mutating the list	Constructing the list
Indexed assignment (<code>lst[i] = ...</code>)	Use of list literals (e.g. <code>[1, 2, 3]</code> , <code>[]</code>)
<code>append</code> method	<code>copy.copy</code> function
<code>extend</code> method	<code>copy.deepcopy</code> function
<code>insert</code> method	<code>+</code> or <code>*</code> operators
<code>pop</code> , <code>reverse</code> , <code>sort</code> methods	Slicing (e.g. <code>lst[x:y:z]</code>)
<code>+=</code> operator	List comprehensions

Math-like operators (OOP)

Math-like Operators		
Expression	Method name	Description
$x + y$	<code>__add__()</code>	Addition
$x - y$	<code>__sub__()</code>	Subtraction
$x * y$	<code>__mul__()</code>	Multiplication
x / y	<code>__div__()</code>	Division
$x == y$	<code>__eq__()</code>	Equality
$x > y$	<code>__gt__()</code>	Greater than
$x >= y$	<code>__ge__()</code>	Greater than or equal
$x < y$	<code>__lt__()</code>	Less than
$x <= y$	<code>__le__()</code>	Less than or equal
$x != y$	<code>__ne__()</code>	Not equal
Sequence Operators		
<code>len(x)</code>	<code>__len__()</code>	Length of the sequence
<code>x in y</code>	<code>__contains__()</code>	Does the sequence <i>y</i> contain <i>x</i> ?
<code>x[key]</code>	<code>__getitem__()</code>	Access element <i>key</i> of sequence <i>x</i>
<code>x[key]=y</code>	<code>__setitem__()</code>	Set element <i>key</i> of sequence <i>x</i> to value <i>y</i>
General Class Operations		
<code>x=myClass()</code>	<code>__init__()</code>	Constructor
<code>print (x), str(x)</code>	<code>__str__()</code>	Convert to a readable string
	<code>__repr__()</code>	Print a Representation of <i>x</i>
	<code>__del__()</code>	Finalizer, called when <i>x</i> is garbage collected

Special Methods

Common Syntax	Special Method Form
$a + b$	<code>a.__add__(b);</code> alternatively <code>b.__radd__(a)</code>
$a - b$	<code>a.__sub__(b);</code> alternatively <code>b.__rsub__(a)</code>
$a * b$	<code>a.__mul__(b);</code> alternatively <code>b.__rmul__(a)</code>
a / b	<code>a.__truediv__(b);</code> alternatively <code>b.__rtruediv__(a)</code>
$a // b$	<code>a.__floordiv__(b);</code> alternatively <code>b.__rfloordiv__(a)</code>
$a \% b$	<code>a.__mod__(b);</code> alternatively <code>b.__rmod__(a)</code>
$a ** b$	<code>a.__pow__(b);</code> alternatively <code>b.__rpow__(a)</code>
$a << b$	<code>a.__lshift__(b);</code> alternatively <code>b.__rlshift__(a)</code>
$a >> b$	<code>a.__rshift__(b);</code> alternatively <code>b.__rrshift__(a)</code>
$a \& b$	<code>a.__and__(b);</code> alternatively <code>b.__rand__(a)</code>
$a \wedge b$	<code>a.__xor__(b);</code> alternatively <code>b.__rxor__(a)</code>
$a b$	<code>a.__or__(b);</code> alternatively <code>b.__ror__(a)</code>
$a += b$	<code>a.__iadd__(b)</code>
$a -= b$	<code>a.__isub__(b)</code>
$a *= b$	<code>a.__imul__(b)</code>
...	...
$+a$	<code>a.__pos__()</code>
$-a$	<code>a.__neg__()</code>
$\sim a$	<code>a.__invert__()</code>
$\text{abs}(a)$	<code>a.__abs__()</code>

Special Methods

<code>abs(a)</code>	<code>a.__abs__()</code>
<code>a < b</code>	<code>a.__lt__(b)</code>
<code>a <= b</code>	<code>a.__le__(b)</code>
<code>a > b</code>	<code>a.__gt__(b)</code>
<code>a >= b</code>	<code>a.__ge__(b)</code>
<code>a == b</code>	<code>a.__eq__(b)</code>
<code>a != b</code>	<code>a.__ne__(b)</code>
<code>v in a</code>	<code>a.__contains__(v)</code>
<code>a[k]</code>	<code>a.__getitem__(k)</code>
<code>a[k] = v</code>	<code>a.__setitem__(k,v)</code>
<code>del a[k]</code>	<code>a.__delitem__(k)</code>
<code>a(arg1, arg2, ...)</code>	<code>a.__call__(arg1, arg2, ...)</code>
<code>len(a)</code>	<code>a.__len__()</code>
<code>hash(a)</code>	<code>a.__hash__()</code>
<code>iter(a)</code>	<code>a.__iter__()</code>
<code>next(a)</code>	<code>a.__next__()</code>
<code>bool(a)</code>	<code>a.__bool__()</code>
<code>float(a)</code>	<code>a.__float__()</code>
<code>int(a)</code>	<code>a.__int__()</code>
<code>repr(a)</code>	<code>a.__repr__()</code>
<code>reversed(a)</code>	<code>a.__reversed__()</code>
<code>str(a)</code>	<code>a.__str__()</code>

Exceptions / Errors

Class	Description
Exception	A base class for most error types
AttributeError	Raised by syntax <code>obj.foo</code> , if <code>obj</code> has no member named <code>foo</code>
EOFError	Raised if “end of file” reached for console or file input
IOError	Raised upon failure of I/O operation (e.g., opening file)
IndexError	Raised if index to sequence is out of bounds
KeyError	Raised if nonexistent key requested for set or dictionary
KeyboardInterrupt	Raised if user types ctrl-C while program is executing
NameError	Raised if nonexistent identifier used
StopIteration	Raised by <code>next(iterator)</code> if no element; see Section 1.8
TypeError	Raised when wrong type of parameter is sent to a function
ValueError	Raised when parameter has invalid value (e.g., <code>sqrt(-5)</code>)
ZeroDivisionError	Raised when any division operator used with 0 as divisor